

Funzioni di elaborazione

$$\text{Elab} : D \rightarrow \text{Env}$$

↳ generato dalla grammatica delle dichiarazioni

$$\forall d \in D. \text{Elab}(d) = p \quad \text{sse} \quad \vdash d \rightarrow_d^* p$$

Equivalenza di dichiarazioni:

$\equiv \subseteq D \times D$ relazione di equivalenza

$$\forall d_1, d_2 \in D. d_1 \equiv d_2 \quad \text{sse} \quad \text{Elab}(d_1) = \text{Elab}(d_2)$$

Esempio

$\underbrace{\text{const } x : \text{int} = 2}_{d_1}$ in $\underbrace{\text{const } y : \text{int} = x + 1;}_{d_3} \underbrace{\text{const } z : \text{int} = x + y;}_{d_4}$
 $d_2 = d_3; d_4$

Dobbiamo elaborare d_1 in d_2

$$\frac{\frac{\vdash d_1 \rightarrow_d d_1' \text{ (oppure } p)}{\vdash d_1 \text{ in } d_2 \rightarrow d_1' \text{ in } d_2} \text{ Regola } d_2 \text{ applicare}}{\vdash \text{const } x : \text{int} = 2 \rightarrow_d [x \mapsto 2]} \text{ primo passo di applicazione}$$

$$\vdash d_1 \text{ in } d_2 \rightarrow_d [x \mapsto 2] \text{ in } d_2$$

Regola ora da applicare :

$$\frac{p_1 \vdash d_2 \rightarrow d_2'}{\vdash p_1 \text{ in } d_2 \rightarrow p_1 \text{ in } d_2'}$$

$\equiv$
 Regola equivalente

$$\frac{p_1 \vdash d_2 \rightarrow_d^* p_2}{\vdash p_1 \text{ in } d_2 \rightarrow p_1 \text{ in } p_2}$$
 eseguo transitivamente l'elaborazione completa di d_2 prima di applicare l'assioma per d_1 in d_2



Elaborazione di $d_2 = d_3; d_4$

Regola da usare:

$$\frac{p_1 \vdash d_3 \rightarrow_d d_3'}{p_1 \vdash d_3; d_4 \rightarrow d_3'; d_4}$$

Versione transitiva:

$$\frac{[p_1 \vdash d_3 \rightarrow_d^* p_3]}{p_1 \vdash d_3; d_4 \rightarrow p_3; d_4}$$

Elaborazione di $d_3 = \text{const } y : \text{int} = x + 1$

Regola da usare:

$$\frac{[p_1 \vdash x+1 \rightarrow_e ?]}{p_1 \vdash \text{const } y:\text{int} = x+1 \rightarrow \text{const } y:\text{int} = ?}$$

$$\frac{p_1 \vdash x \rightarrow 2}{p_1 \vdash x+1 \rightarrow 2+1} \quad p_1(x) = 2 \quad p_1 \vdash 2+1 = 3$$

$$\frac{p_1 \vdash x+1 \rightarrow^* 3}{p_1 \vdash \underbrace{\text{const } y:\text{int} = x+1}_{d_3} \rightarrow \underbrace{\text{const } y:\text{int} = 3}_{d_3'}} \rightarrow p_1 \vdash \text{const } y:\text{int} = 3 \rightarrow \overbrace{[y \mapsto 3]}^{p_3}$$

Riscriviamo la regola

$$\frac{p_1 \vdash d_3 \rightarrow^* p_3}{p_1 \vdash d_3; d_4 \rightarrow p_3; d_4}$$

Regola successiva transitiva:

$$\frac{[p_1[p_3] \vdash d_4 \rightarrow^* p_4]}{p_1 \vdash p_3; d_4 \rightarrow p_3; p_4}$$

Elaboriamo $d_4 = \text{const } z:\text{int} = x+y$ in $p_1[p_3] = [x \mapsto 2, y \mapsto 3]$

Regola da applicare:

$$\frac{p_1[p_3] \vdash x+y \rightarrow^* k}{p_1[p_3] \vdash \text{const } z:\text{int} = x+y \rightarrow \text{const } z:\text{int} = k}$$

Cerchiamo k

$$\left[\begin{array}{l} \frac{p_1[p_3] \vdash x \rightarrow 2}{p_1[p_3] \vdash x+y \rightarrow 2+y} \quad p_1[p_3](x) = 2 \\ \frac{p_1[p_3] \vdash y \rightarrow 3}{p_1[p_3] \vdash 2+y \rightarrow 2+3} \quad p_1[p_3](y) = 3 \\ p_1[p_3] \vdash 2+3 \rightarrow 5 \end{array} \right.$$

$$\frac{p_1[p_3] \vdash x+y \rightarrow^* 5}{p_1[p_3] \vdash \text{const } z:\text{int} = x+y \rightarrow \text{const } z:\text{int} = 5}$$

App. assioma

$$\frac{p_1[p_3] \vdash \text{const } z:\text{int} = 5}{\overbrace{[z \mapsto 5]}^{p_4}}$$

Riscriviamo l'ultima regola usata per d_2

$$\frac{p_1[p_3] \vdash d_1 \rightarrow^* p_4}{p_1 \vdash p_3; d_1 \rightarrow p_3; p_4}$$

Applichiamo l'assioma

$$p_1 \vdash p_3; p_4 \rightarrow p_3[p_4] = p_2 = [y \mapsto 3, z \mapsto 5]$$

$d_2 \rightarrow^* p_3[p_4]$

Risolviamo *

$$\frac{p_1 \vdash d_2 \rightarrow^* p_2}{\vdash p_1 \text{ in } d_2 \rightarrow p_1 \text{ in } p_2}$$

Applichiamo l'assioma

$$\vdash p_1 \text{ in } p_2 \rightarrow p_2$$

$$\Rightarrow \vdash d_1 \text{ in } d_2 \rightarrow^* p_2 = [y \mapsto 3, z \mapsto 5]$$

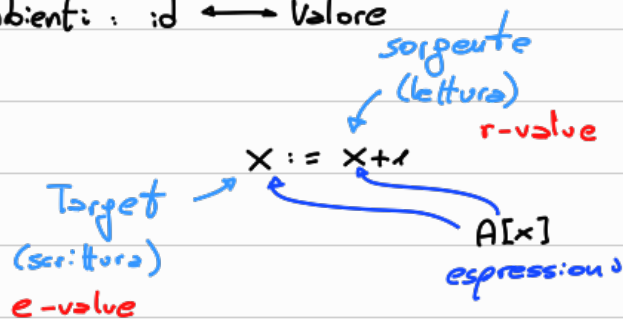
[0: 33 / 24]

Espressioni senza id + espressioni con id + dichiarazioni di costante
 tipo ambiente statico identificatore $x \mapsto \text{valore costante}$

no dobbiamo arrivare ad introdurre identificatori variabile

Ambienti: $\text{id} \longleftrightarrow \text{Valore}$

richiede l'introduzione della MEMORIA e dei COMANDI



- Diversi oggetti: con lo stesso nome (ricorsione, ridef. di var. ...)
- Diversi nomi per lo stesso oggetto (puntatori, aliasing ...)
- Nomi e valori che cambiano durante l'esecuzione

=> LOCAZIONE



Memoria → tenere traccia delle trasformazioni: effettuate dall'esecuzione di programmi

Localizzazione $\rightarrow$ Viene creata dalle dichiarazioni $\rightarrow$ di compilazione $\rightarrow$ creazioni a tempo

- Viene creata dai comandi = tempo di esecuzione (nei linguaggi dinamici/interpretati)
- Ha un tempo di vita
- Viene usata per contenere/riferire a oggetti che possono cambiare durante l'esecuzione
- Considerate illimitate

[0:53:00]

Identifieri constant: (Non usano la memoria)

⇒ vengono associati 2 valori che non cambiano durante l'esecuzione

→ sono legami che restano nell'ambiente dinamico

const $x \xrightarrow{\text{legame di binding}} 2 \rightarrow \text{Ambiente}$

$$\pi \rightarrow 3,14$$

Variabili (ling. imperativi)

Sono identificate da due legami: uno nell'ambiente

$x \rightarrow \boxed{}$ locazione nuovo valore denotabile

uno nella memoria

$\square \rightarrow 2$ il cui valore memorizzabile può cambiare

var x $\xrightarrow{\text{legame}}$ $\rightarrow$

$\downarrow$
memoria

In Imp $\rightarrow$ valori esprimibili (elem. primitivi di Exp) BUN

valori memorizzabili: BVN

↓
valori denotabili: $B \vee \neg U$ (possono essere legati da un $\forall$)
(locazioni)

D: viene modificata aggiungendo la dichiarazione di variabile

$$D: D \rightarrow \text{nil} \mid p \mid \text{const } x:z=e \mid D; D \mid D \text{ in } D$$

`var x:z=c` dichiara una variabile di nome `x` di tipo `z` e valore iniziale descritto da `c`

$$\begin{cases} FI(\text{var } x:z=e) = FI(e) \\ DI(\text{var } x:z=e) = DI(x) \end{cases}$$

[4:05:00]

~~$x := e$~~ se x è const

Dobbiamo aggiungere i tipi $DType$ per le locazioni:

$$DType = \{int, bool, \underbrace{intloc, boolloc}_{\substack{\text{i tipi delle locazioni} \\ zloc}}\}$$

Regola statica per var:

$$\frac{\Delta \vdash e : z}{\Delta \vdash \text{var } x : z = e : [x \mapsto zloc]}$$